



Học Sâu (Deep Learning)

Chương 2: Các khối xây dựng toán học của Mạng Nơ-ron

Đại học Đông Á
August 14, 2026

Nội dung chương

1. Ví dụ đầu tiên về
Mạng Nơ-ron (MNIST)



2. Biểu diễn dữ liệu bằng
Tensor



3. Phép toán Tensor
(Tensor Operations)



4. Tối ưu hóa Dựa trên
Gradient



5. Đồ thị Tính toán &
Lan truyền ngược



6. Tổng kết



Ví dụ kinh điển: Tập dữ liệu MNIST

Bài toán: Phân loại ảnh các chữ số viết tay (từ 0 đến 9).

Dữ liệu: 60.000 ảnh huấn luyện,
10.000 ảnh kiểm tra.

Kích thước mỗi ảnh: 28×28 pixels (ảnh xám đen trắng).

Trong học máy:

Phân loại (Category) gọi là Lớp (Class).

Dữ liệu đầu vào gọi là Mẫu (Sample).

Kết quả của mẫu gọi là Nhãn (Label).



Hình 2.1: Một số mẫu ảnh từ MNIST



Tải dữ liệu MNIST trong Keras

Tập dữ liệu MNIST được tải sẵn trong Keras, dưới dạng 4 mảng NumPy.

```
from keras.datasets import mnist

(train_images , train_labels), (test_images , test_labels) = mnist.load_data ()
```

- Tập huấn luyện (Training set): train_images và train_labels
- Tập kiểm tra (Test set): test_images và test_labels
- Hình ảnh được mã hóa dưới dạng mảng NumPy, nhãn là các số từ 0 đến 9.

Khám phá hình dạng dữ liệu

Mảng NumPy là thư viện phổ biến để tính toán số. Nó được sử dụng làm định dạng lưu trữ trao đổi dữ liệu. Hãy xem xét cấu trúc của tập huấn luyện:

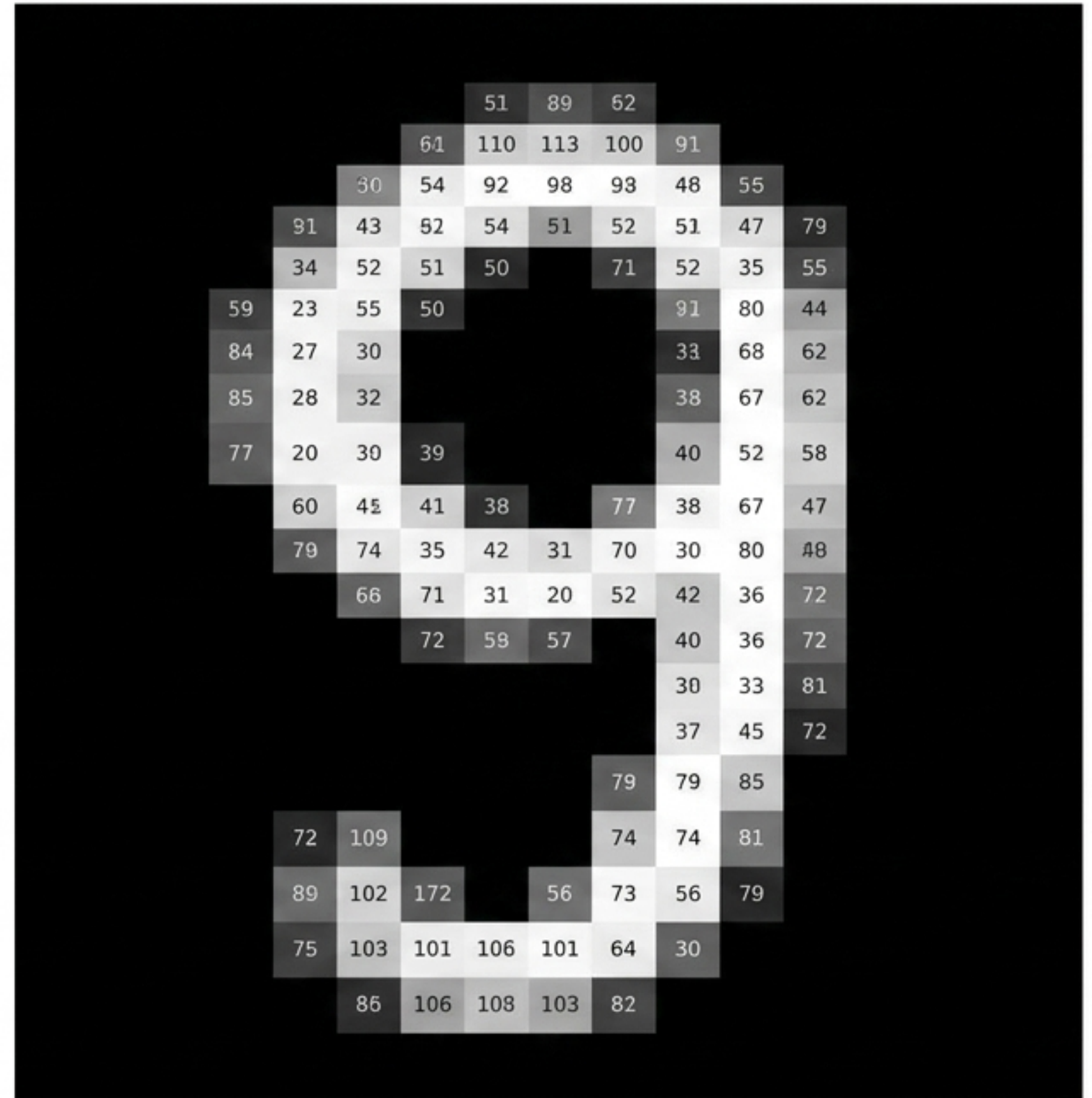


```
>>> train_images.shape
(60000 , 28, 28)
>>> len(train_labels)
60000
>>> test_images.shape
(10000 , 28, 28)
```

Dữ liệu huấn luyện gồm 60.000 mẫu, mỗi mẫu là một ma trận 28× 28.

Xem xét một mẫu dữ liệu (Sample)

- Mỗi bức ảnh là một ma trận số nguyên.
- Giá trị từ 0 (trắng) đến 255 (đen).
- Ở đây chúng ta hiển thị mẫu thứ 4 trong tập huấn luyện (chữ số '9').



Hình 2.2: Biểu diễn ma trận của một ảnh



Kiến trúc của mô hình mạng nơ-ron

Khối xây dựng cốt lõi của mạng nơ-ron là **lớp (layer)**.

Các lớp **trích xuất các biểu diễn** từ dữ liệu được cung cấp.

Lớp **Dense** là lớp được kết nối đầy đủ (**fully connected**).

Lớp cuối cùng là **softmax 10 chiều**, trả về mảng 10 điểm xác suất.

```
import keras
from keras import layers

model = keras.Sequential(
    [
        layers.Dense (512, activation="relu"),
        layers.Dense (10, activation="softmax"),
    ]
)
```



Biên dịch mô hình (Compile)

```
model.compile(  
    optimizer="adam",  
    loss="sparse_categorical_crossentropy",  
    metrics=["accuracy"],  
)
```



Tiền xử lý dữ liệu

Biến đổi ảnh 28× 28 thành vector 784 chiều, đưa giá trị về khoảng [0, 1].

```
train_images = train_images.reshape((60000 , 28 * 28))  
train_images = train_images.astype("float32") / 255
```

Huấn luyện (Training)



Quá trình huấn luyện (khớp dữ liệu) trong Keras được thực hiện thông qua **model.fit()**.



Đánh giá (Evaluation)



Sau khi huấn luyện, chúng ta đánh giá mô hình trên tập kiểm tra:

```
model.fit(train_images , train_labels , epochs=5,  
          batch_size=128)
```

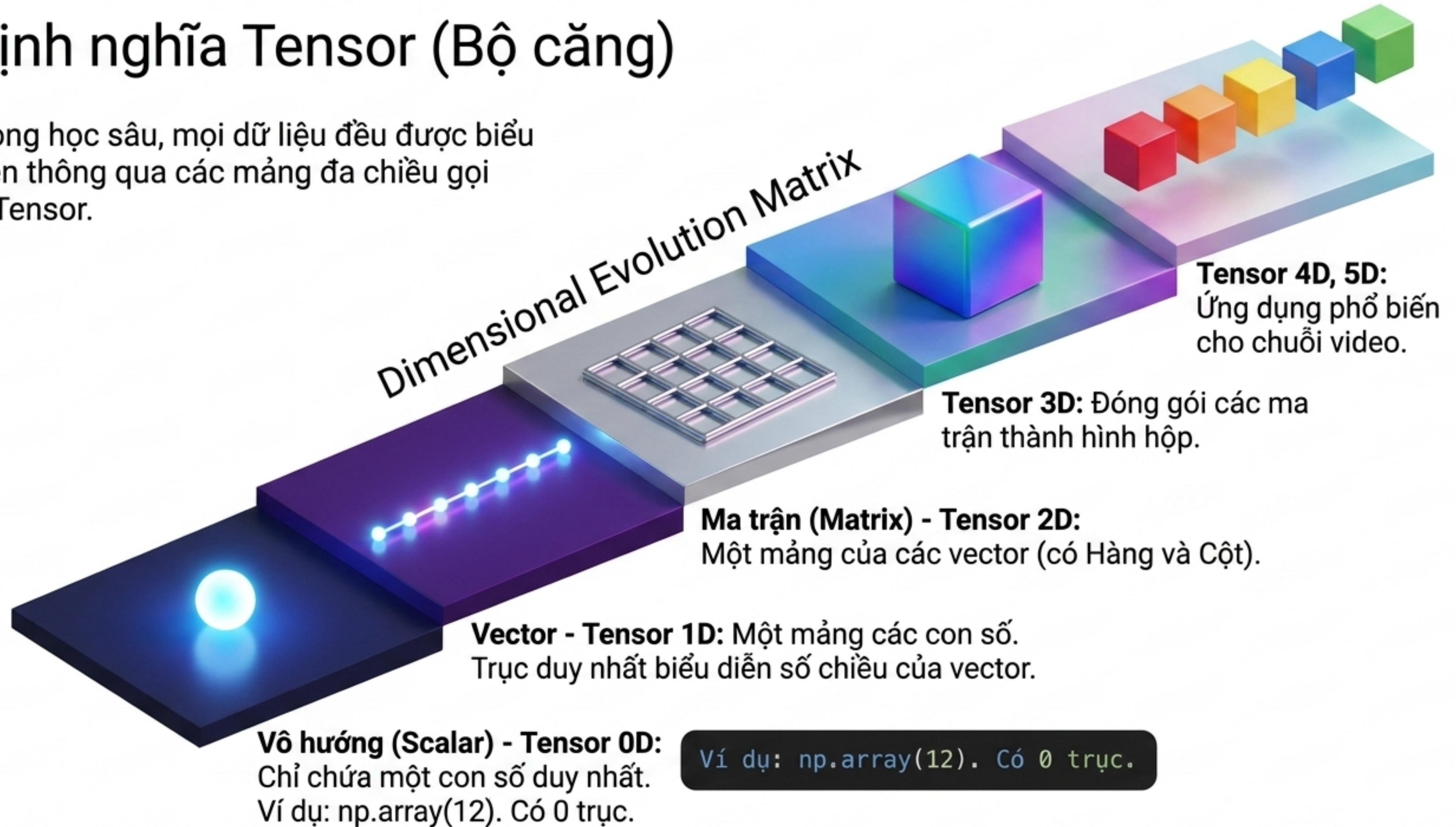
```
test_loss , test_acc =  
model.evaluate(test_images , test_labels)  
print(f"test_acc: {test_acc}") # Khoảng 97.8%
```



Hiện tượng mô hình hoạt động kém hơn trên dữ liệu mới so với dữ liệu huấn luyện gọi là **Overfitting (Quá khớp)**.

Định nghĩa Tensor (Bộ căng)

Trong học sâu, mọi dữ liệu đều được biểu diễn thông qua các mảng đa chiều gọi là Tensor.



Các thuộc tính chính của Tensor

Một tensor được xác định bởi ba thuộc tính cốt lõi:



1. Số trục (thứ hạng - rank):

Được biểu thị bằng ndim trong NumPy. Một ma trận có 2 trục, Tensor 3D có 3 trục.



2. Hình dạng (shape):

Là một tuple chứa số nguyên mô tả số chiều dọc theo mỗi trục. Ví dụ: (60000, 28, 28).



3. Kiểu dữ liệu (dtype):

Loại dữ liệu chứa trong tensor (float32, uint8, float64, bool v.v...).

Thao tác với Tensor: Cắt (Slicing) và Khái niệm Lô (Batch)



Cắt Tensor (Slicing): Lựa chọn phần tử cụ thể dọc theo các trục.

```
my_slice = train_images [10:100] # Chọn từ index 10 đến 99  
# Tương đương: train_images [10:100 , :, :]  
# Cắt ở giữa bức ảnh 14x14:  
my_slice = train_images[:, 7:-7, 7:-7]
```



Khái niệm Lô (Batch): Mô hình học sâu xử lý dữ liệu theo các đợt nhỏ. Trục đầu tiên (trục 0) thường được gọi là trục lô (batch axis).

```
batch = train_images[:128] # Lô đầu tiên gồm 128 mẫu
```

Ví dụ thực tế: Vector và Dữ liệu Chuỗi thời gian

- Dữ liệu Chuỗi thời gian (Tensor 3D)
- Hình dạng: (mẫu, thời gian, đặc trưng)
- Trục thời gian luôn là trục thứ hai (index 1).
- Ví dụ: Giá cổ phiếu mỗi phút (hiện tại, cao nhất, thấp nhất) trong 390 phút, 250 ngày giao dịch → Tensor (250, 390, 3).



Hình 2.3: Tensor 3D cho dữ liệu chuỗi thời gian

Ví dụ thực tế: Dữ liệu Ảnh

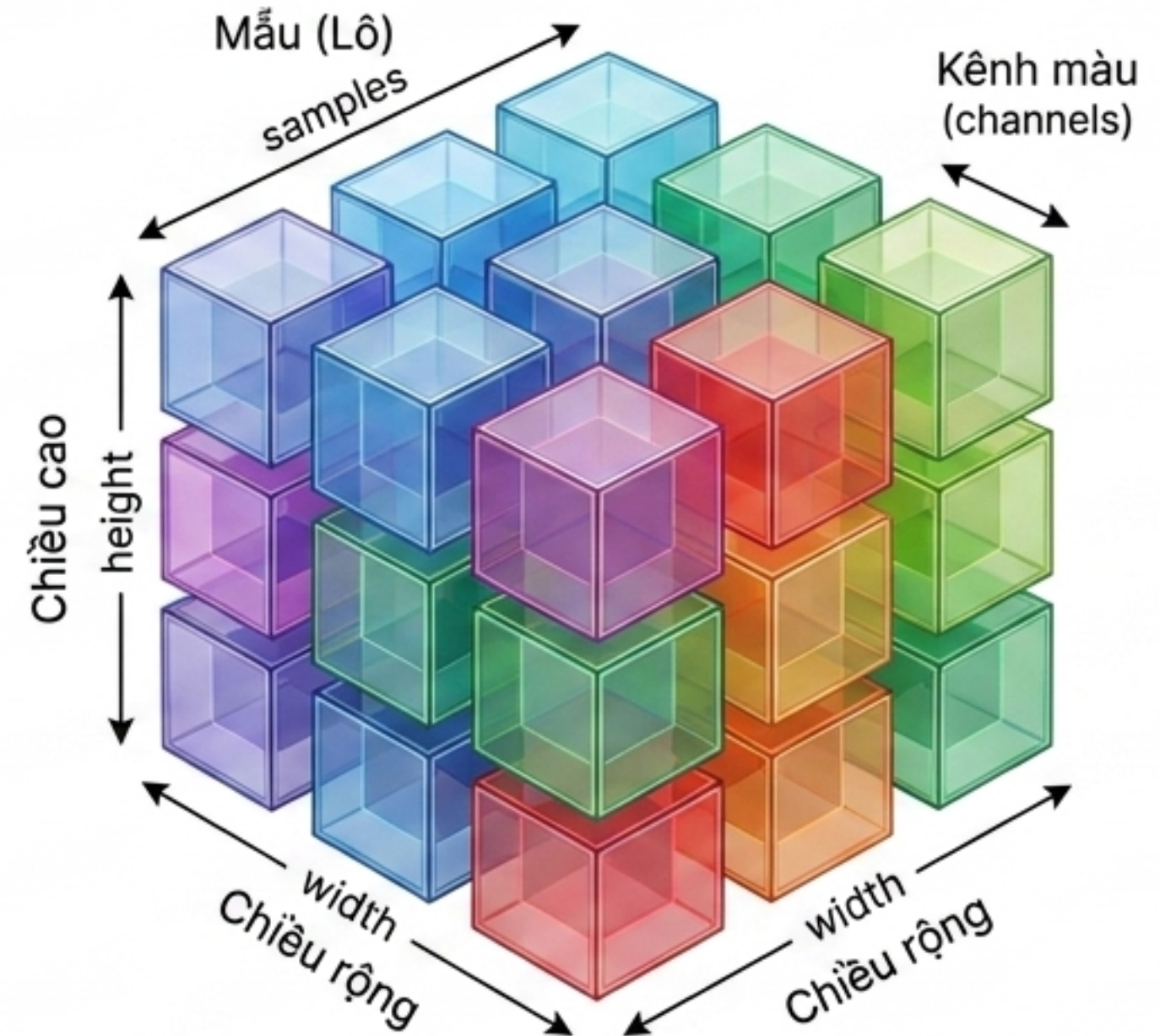


Ảnh kỹ thuật số thường có 3 chiều: Chiều cao, Chiều rộng và Kênh màu.

Lô ảnh (Batch of images): Tạo thành một Tensor 4D.

Hình dạng: (mẫu, chiều cao, chiều rộng, kênh màu)

Ví dụ: Lô 128 bức ảnh màu cỡ 256×256 có hình dạng là $(128, 256, 256, 3)$.



Hình 2.4: Tensor 4D biểu diễn dữ liệu ảnh màu

Ví dụ thực tế: Dữ liệu Video



Dữ liệu video là trường hợp phức tạp nhất trong thực tế, sử dụng Tensor 5D.

Một video là một chuỗi các khung hình (frames), mỗi frame là ảnh màu 3D.

Hình dạng của tensor: (mẫu, khung hình, chiều cao, chiều rộng, kênh màu)

Ví dụ: Một lô 4 clip YouTube 60 giây (144×256 pixels, 4 fps) có 240 khung hình sẽ có hình dạng (4, 240, 144, 256, 3).

Kích thước này rất lớn
(tương đương ~ 425 MB
cho kiểu float32).

